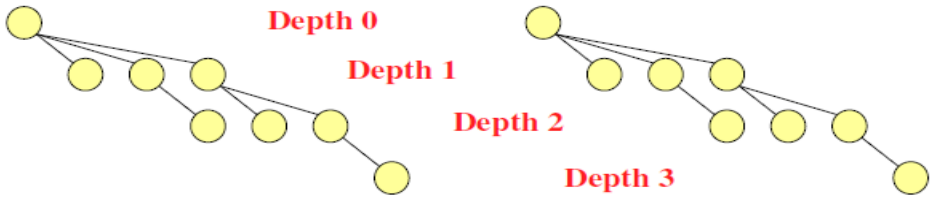
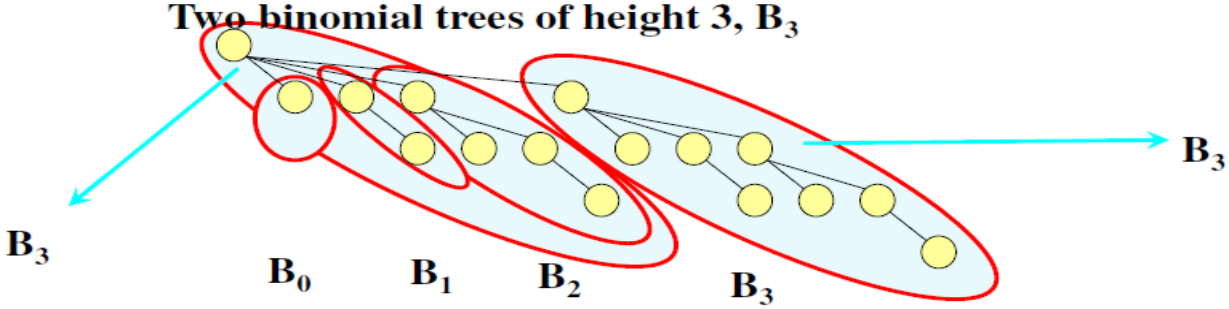
Q1. (2013)10 pts)
How many nodes does the binomial heap BH={ B_2 B_4 B_7 B_{10} B_{13} B_{17} } have?

Binomial Trees



Two binomial trees of height 3, B_3



A binomial tree of height 4, B_4

- A *binomial tree* B_k has
  - a height of k;
  - $n = 2^k$ **nodes (!!!)**;
  - $k+1$ depth levels ranging within $0,\ldots,k$.
  - $\binom{k}{d} = \frac{k!}{d!\ (k-d)!}$ nodes at *depth level* d.
  - a root and a B_0, B_1, B_2, B_3,..., B_{k-1} connected to it in respective order (see slide 96!).

|BH|=4+16+128+1024+8192+131072=140436

Q2. (2011) (10+10+10=30pts)

The two largest binomial trees in an ***n***-node *binomial heap BH* is B_{12} and B_8.

Binomial Heaps

- *BHs* differ from other heap structures in that
  - a BH is *not a heap-ordered tree* but rather *a collection of heap ordered trees*, ***a forest***.
  - each heap-ordered tree is of a constrained form known as a *binomial tree*.
- Each binomial tree in a BH obeys *min-heap* or *heap order* property.
- There is *at most one* B_k of *each height* k in a *BH*.
- In the next slide, we see an 11-node BH.

a) (10pts)
Give a range for *n* such that its lower and upper limits specify the least and most crowded binomial heaps, respectively. In other words, how many nodes, at least and at most, may such a BH composed of?

$BH_{min} = B_8\ B_{12}$
$\Rightarrow n_{min} = 256+4096 = 4352$ nodes
$BH_{max} = B_0\ B_1\ B_2\ B_3\ B_4\ B_5\ B_6\ B_7\ B_8\ B_{12}$
$\Rightarrow n_{max} = 1+2+4+8+16+32+64+128+256+4096 = 4607$ nodes

$\Rightarrow n \in \{4352,\ldots,4607\}$

b) (10pts)
How many nodes are there at the most crowded depth level of the most crowded binomial tree in the above BH?

Most crowded binomial tree is B_{12}. The most crowded depth level is $dl = 6$.

In order to find the total number of nodes in depth d in a binomial tree with height k we need to use the following formula:

$$\binom{k}{d} = \frac{k!}{d!\ (k-d)!} \text{ nodes at } \textit{depth level } d.$$

$$dl_6 = \binom{12}{6} = \frac{12x11x10x9x8x7}{1x2x3x4x5x6} = 924 \text{nodes}$$

c) (10pts)
Suppose the most crowded BH in (a) and that the node with the minimum key is at the root of B_8. Exactly, how many comparisons does it take to find the minimum key?

$BH_{max} = B_0\ B_1\ B_2\ B_3\ B_4\ B_5\ B_6\ B_7\ B_8\ B_{12}$

BH_{max} has 10 binomial trees. Considering our in-class discussions the roots of the binomial trees in the BH form a linked list. Hence B_8 is the 9th tree of the BH. The minimum key at the root of B_8 takes nine comparisons exactly to find.

Q3. (2005) Consider a binomial queue with 9874 nodes.

i. (5pts) Which binomial trees is it composed of?
ii. (5pts) How many nodes does the **largest tree** have in **its most crowded depth level(s)**? Which level(s) are they?

i. $B_{13}\ B_{10}\ B_9\ B_7\ B_4\ B_1$
ii. $|BH| = 2+16+128+512+1024+8192 = 9874$

$2^{15} = 32768$
$2^{14} = 16384$
$2^{13} = 8192$

Number of nodes in the **most crowded depth level of** the **largest tree** $= \binom{k}{\lceil k/2 \rceil}\binom{13}{7} = 1716$

Q4. (2013) (5+10+10+15=40pts)
Suppose a binomial heap BH is devised by merging two binomial heaps each with $n=2^k-1$ keys.

a) How many nodes, in terms of n, does BH contain?
|BH|=n+n=2n

b) What binomial trees is BH composed of?

$$\sum_{k=1}^{n} r^{k-1} = 1+r+r^2+r^3+\ldots+r^{n-1} = \frac{1-r^n}{1-r},\ (r \neq 1)$$

BH_1: $\{B_0\ B_1\ B_2\ \ldots\ B_{k-1}\}$: $2^0+2^1+2^2+\ldots+2^{k-1} = \frac{1-2^k}{1-2} = -(1-2^k) = 2^k-1$

BH_2: $\{B_0\ B_1\ B_2\ \ldots\ B_{k-1}\}$
BH: $\{B_1\ B_2\ B_3\ \ldots\ B_{k-1}\ B_k\}$

c) **In terms of *n***, how many binomial trees is BH **exactly** composed of?

BH is composed of *k* binomial trees.
$n=2^k-1$;
$k=\log_2(n+1)$;

d) How many keys are there, **in terms of *n***, in the most crowded depth level of the largest binomial tree?

The largest binomial tree is B_k. It has 2^k keys and $k+1$ depth levels ranging from *0* to *k*. We know that a binomial tree has $\binom{k}{d}$ keys at its dth depth level. We also know that the most crowded depth level of a binomial tree B_k is $\lceil k/2 \rceil$; hence its name ***binomial*** tree.

Thus, the most crowded depth level of the largest binomial tree B_k has $\binom{k}{\lceil k/2 \rceil}$ keys.

Further from (c), $k=\log_2(n+1)$. Therefore,

$$N = \binom{k}{\lceil k/2 \rceil} = \binom{\log_2(n+1)}{\left\lceil \frac{\log_2(n+1)}{2} \right\rceil}$$

Q5. (2014) (5 + 10 pts)
How many comparisons, **in terms of *n***, does it take to find the node with the minimum key in a binomial heap with $n=2^k$ -3 elements
a) in the worst case, and
b) in the average case
Answer:

BTs: (i.e., B_0, B_2, B_3, …, B_{k-1}).
We have k-1 BTs.
$k=\log_2(n+3)$
$k-1=\log_2(n+3)-1$

Since we look for the minimum key, it must be at the root of one of the $\log_2(n+3)-1$ binomial trees. So, we are searching for the minimum key at the linked list formed by the

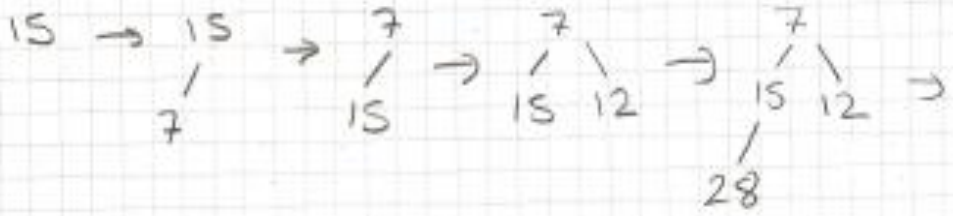
a) $\log_2(n+3)-1$ comparisons in the worst case, and
b) $(\log_2(n+3) -1) /2$ comparisons in the average case.

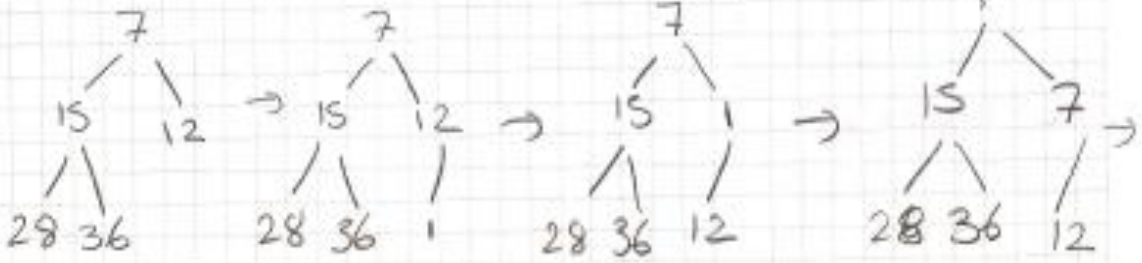
Q6. 2013-Quiz

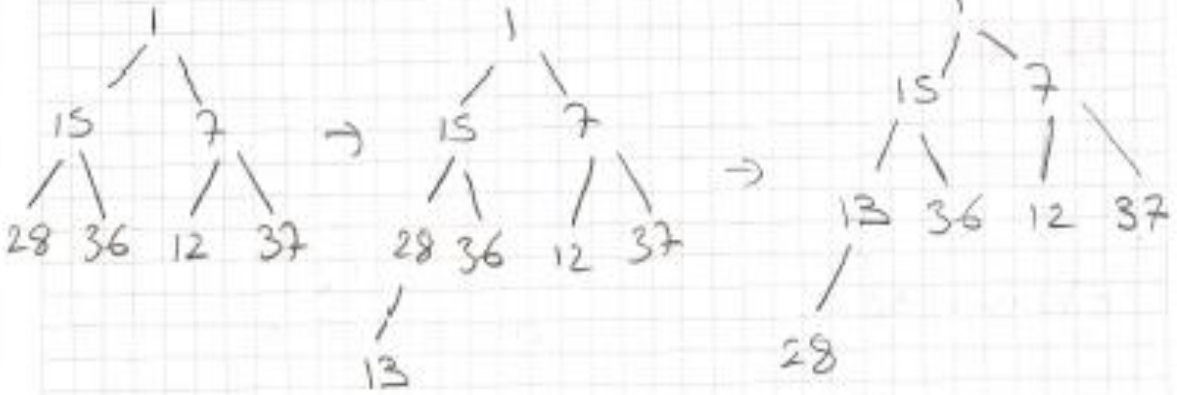
2. Construct a binary min-heap using the following sequence of numbers.

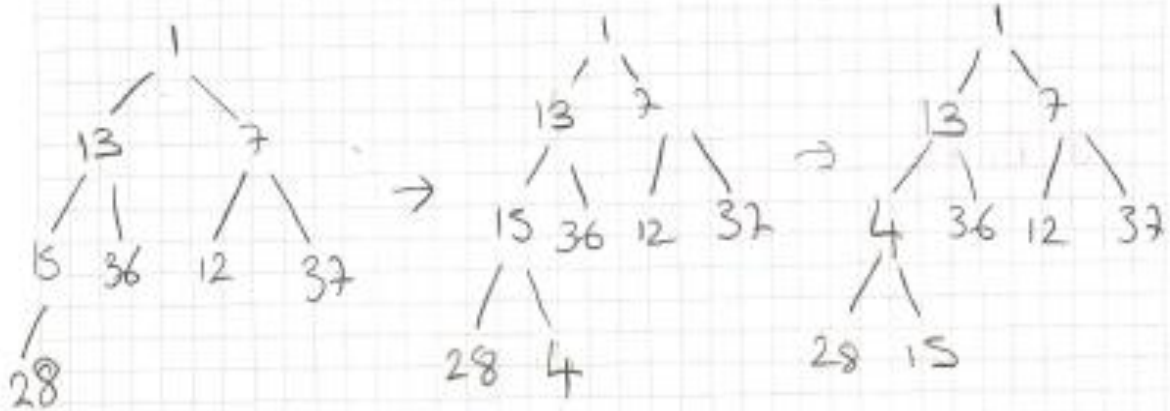
15, 7, 12, 28, 36, 1, 37, 13, 4, 67, 49

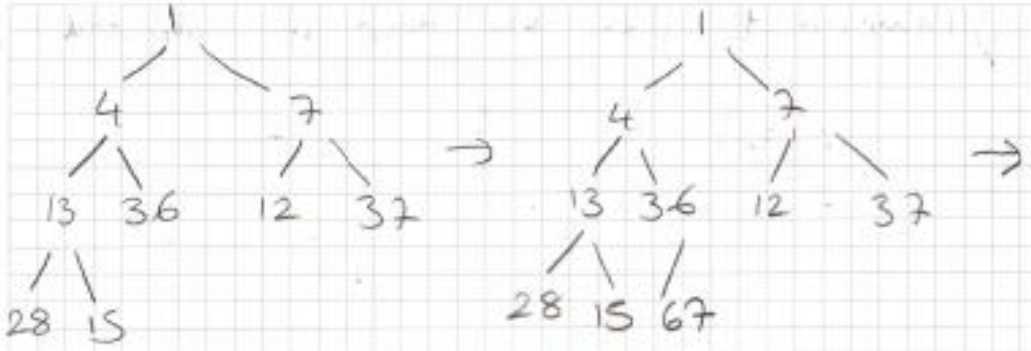
a)

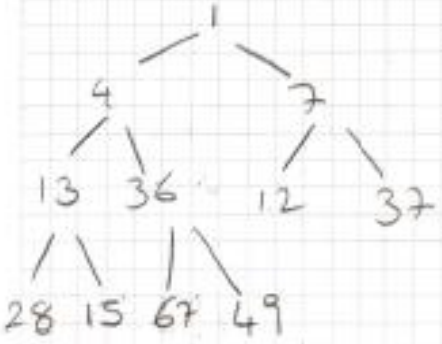




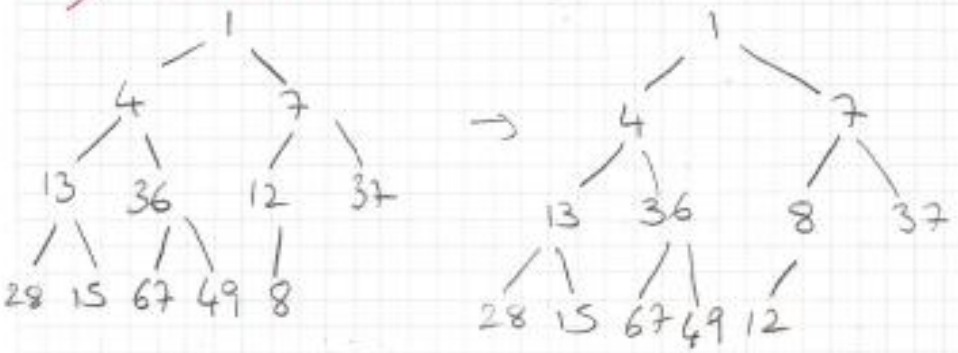




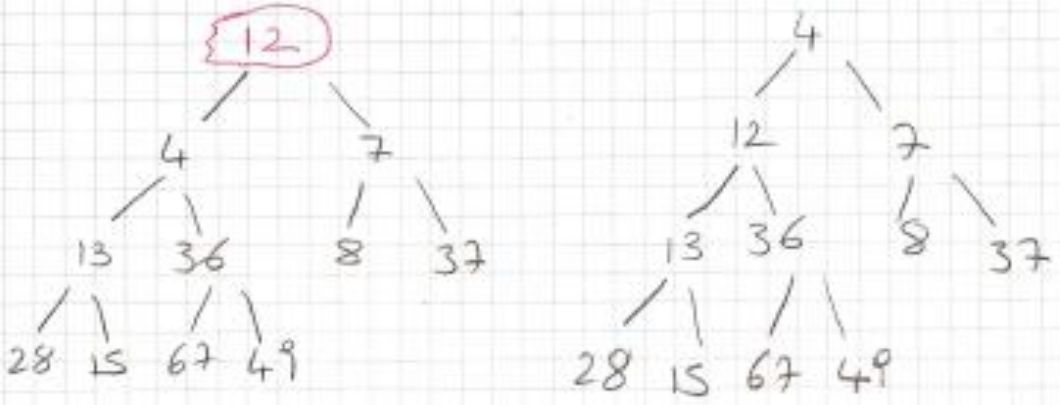




b) Insert 8



c) Remove 1 from the heap you obtained in b.



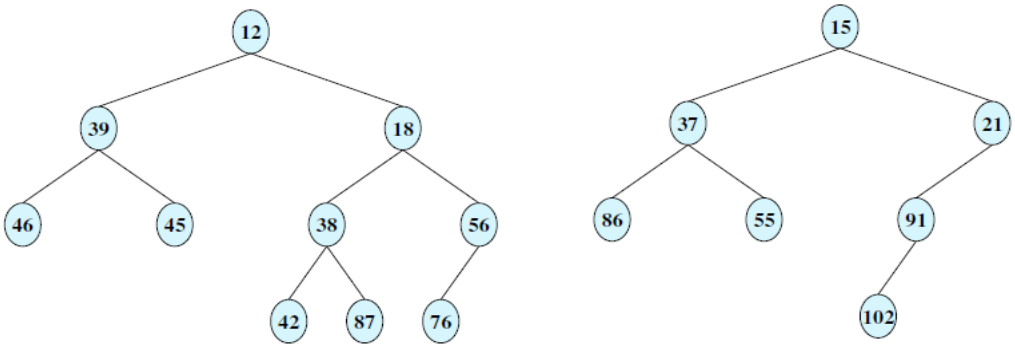
Q7. (2015)(10 pts)
Suppose you have two leftist heaps L_1 and L_2 with *n* elements in total. How many comparisons does it take exactly to merge L_1 and L_2 in terms of n?

a) at most (worst case), and
b) at least (best case)

Non-Recursive Merging Algorithm: First Pass

- *Two passes*
- *First pass*:
  - arrange the nodes on the *right-most path* of both LHs in *sorted order*, keeping their respective left children;
  - create a new tree from the two LHs;
    - sort and arrange the nodes above,
    - make them the right path of the new tree.

Merging Example

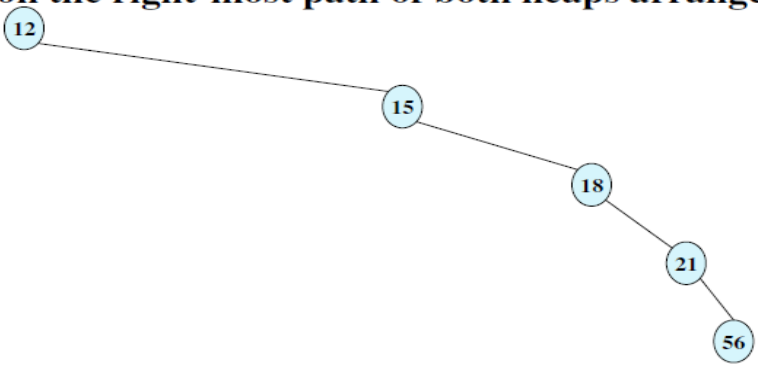


H1 H2

Merging Example: First Pass



- **Nodes on the right-most path of both heaps arranged!**



- **A new tree made with the above as its right-most path!**

a) Worst case:
The worst case is the one with the largest number on the right-most path.
This should be a perfect binary tree.
Each such leftist heap would have exactly *n/2* nodes

$n/2 = 2^{k-1} - 1$

where *k-1* nodes at its right-most path.

Remember number of comparison for merging two sorted lists is: n+m-1

The number of comparisons, *N*, would, hence, add up to *2(k-1)-1=2k-3*. In terms of *n*, this would be equal to

$((n/2)+1) = 2^{k-1}$
$n+2=2^{k}$
$k = \log_2(n+2)$

$$k = \log_2(n+2)$$
$$\Rightarrow N = 2k-3 = 2\log_2(n+2)-3$$

b) Best case:
Regardless of the number of nodes n, both leftist heaps may have only their root on their right most path which result in a single comparison to merge them. Hence in the best case, the exact number of comparisons is ***N=1***.

Q8. (2013) (8+8=16 pts) (HOMEWORK)
Consider two binomial queues BQ_1 and BQ_2 with $n=2^k$ and $m=2^l$ elements, respectively, where $k<l$.

a) One *DeleteMin* is performed at each binomial queue and then they are merged to BQ_M. How long does it take, in average, to find the minimum key in BQ_M?

After one DeleteMin at both BQs, BQ_1 and BQ_2 are composed of the following binomial trees:

$n-1=2^k-1$
$m-1=2^l-1$
BQ_1: $\{B_0\ B_1\ B_2\ \cdots\ B_{k-1}\}$
BQ_2: $\{B_0\ B_1\ B_2\ \cdots\ B_{k-1}\ \cdots\ B_{l-1}\}$

After merging BQ_1 and BQ_2 we get BQ_M which is composed of the following binomial trees:

BQ_M: $\{B_1\ B_2\ \cdots\ B_{k-1}\ B_k\ B_{k+1}\ldots\ B_l\}$

$t_{ave} = \frac{l+1}{2};$ remember average search time for LLs

b) Suppose that BQ_1 and BQ_2 are first merged, and then two *DeleteMin*s are performed. Suppose further that both minimum keys are at B_k. The second minimum key, located among the children of the root, is at the root of the most crowded binomial tree. What is your answer to the same question in part (a)?

In this scenario, merging BQ_1: $\{B_k\}$ and BQ_2: $\{B_l\}$ into BQ_M we get BQ_M: $\{B_k\ B_l\}$.

Removing the first minimum key we get

$$BQ'_M : \{B_0\ B_1 \cdots B_{k-1} B_l\}$$

The second key is at the root of the most crowded binomial tree, i.e., B_{k-1}. Removing the second minimum key, and thus, breaking B_{k-1} up into its sub-binomial trees, we get BQ_{rest}: $\{B_0\ B_1\ B_2 \cdots B_{k-2}\}$. Otherwise remain from BQ'_M $\{B_0\ B_1\ B_2 \cdots B_{k-2}\ B_l\}$. Hence, after the second *DeleteMin*, we get

$$BQ''_M : \{B_0\ B_1\ B_2 \cdots B_{k-2}\ \{B_0\ B_1 \cdots B_{k-2}\}\ B_l\}$$

Since there may exist only a unique B_h from any height h, we restore the heap property of BQ''_M by merging BQ_{rest} into $\{B_0\ B_1\ B_2 \cdots B_{k-2}\ B_l\}$ where we get

$$BQ''_M : \{B_1\ B_2 \cdots B_{k-2}\ B_{k-1}\ B_l\}$$

Q9. (20pts) (HOMEWORK)

a) (5pts) Build <u>two</u> binomial queues using the two number sequences below. **<u>Show both queues after *each* insert operation</u>**!

11, 23, 6, 45, 18, 96, 63, 75, 57, 43;

9, 16, 44, 11, 26, 7, 13, 22

b) (5pts) Apply one `DeleteMin` operation in the first and two `DeleteMin` operations in the second binomial queue! Show the queues after each operation

c) (5pts) Merge the two resulting queues after the operations in (b)!

d) (5pts) Consider a binomial queue with 5265 nodes.

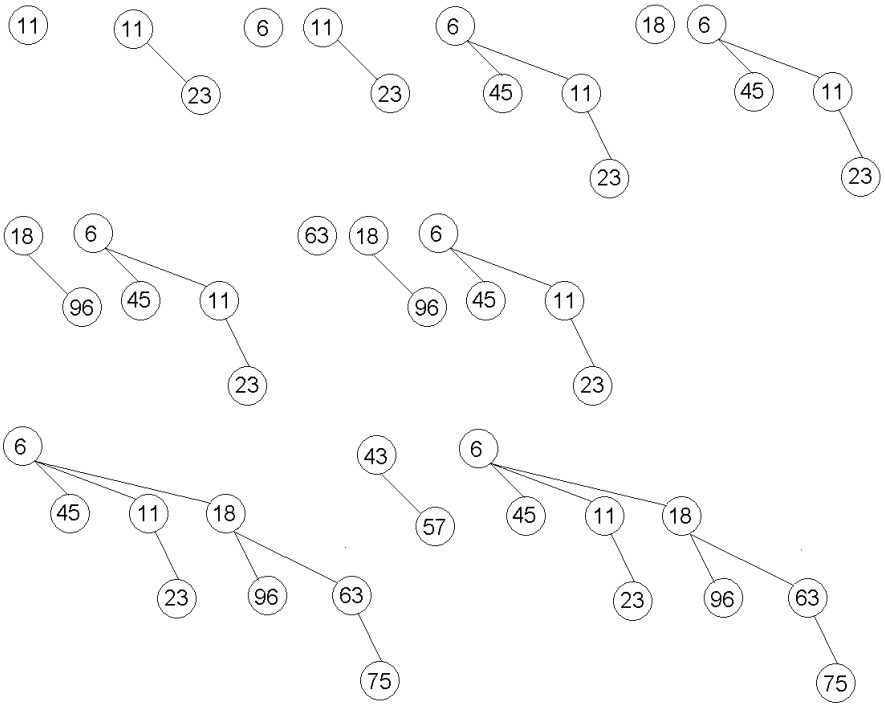
iii. (2.5pts) Which binomial trees is it composed of?

iv. (2.5pts) How many nodes does the **largest tree** have in **its most crowded depth level**?

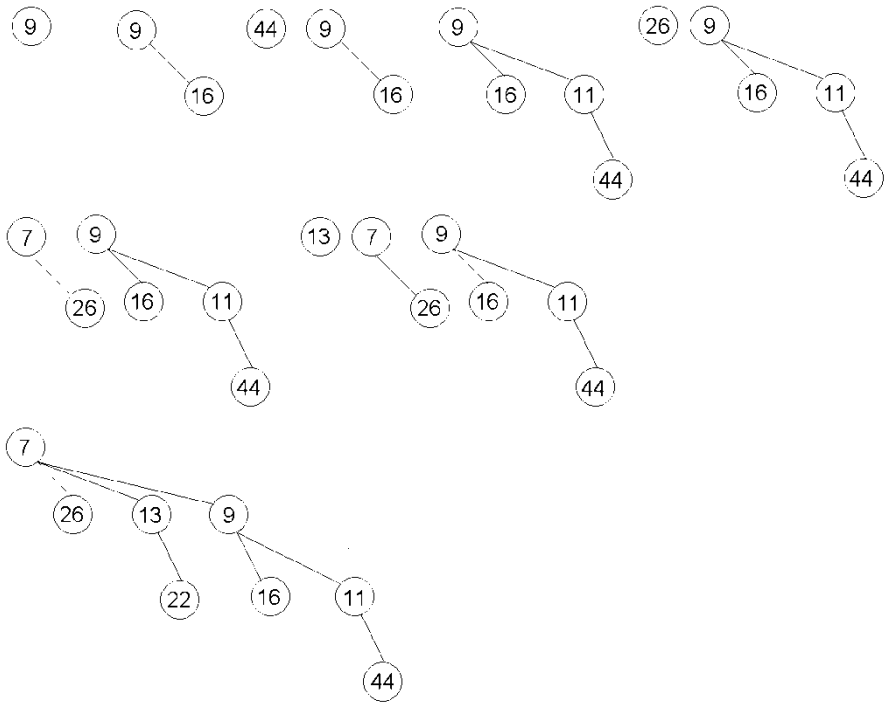
Answer:

a)

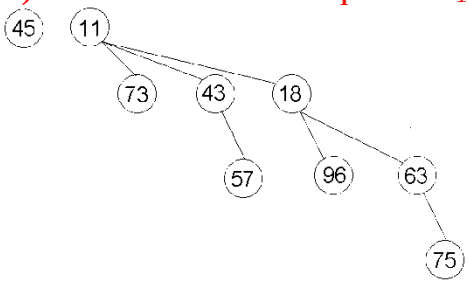
The first binomial queue: H_1



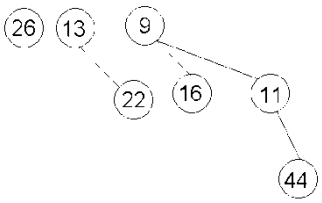
The second binomial queue: H_2



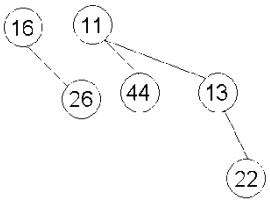
b) The first binomial queue H_1' after *DeleteMin*



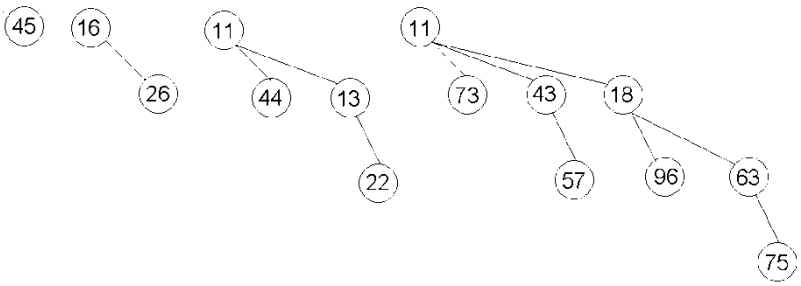
H_2' after first *DeleteMin*



H_2'' after second *DeleteMin*



c) Resulting binomial queue H_M at the end of the merging



d)

i. $B_{12}\ B_{10}\ B_7\ B_4\ B_1$

ii. $\binom{12}{6} = 924$